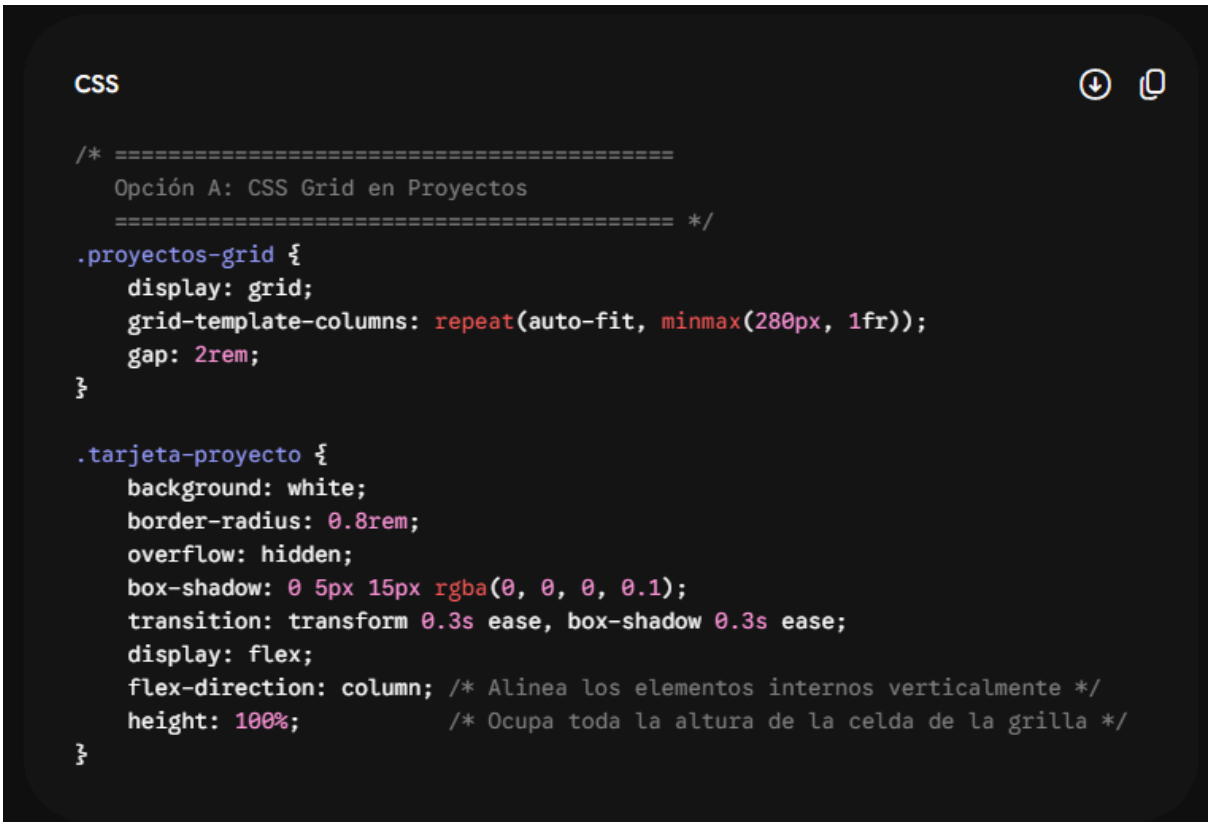
Mateo Ruppel

a mi portfolio web base ya entregado elegi de entre las 4 opciones de mejora, css grid y api publica con fetch y async/await

elegi el chat web gemin de google

para el css grid empeze con el prompt inicial de “tengo una seccion en mi portfolio web donde uso flexbox o flex wrap como puedo migrar esto a css grid sin necesidad de media queries manuales para las columnas?”
me respondio que reemplazando la propiedad display y la estructura en el archivo luego en el prompt de iteracion le dije
“como ajusto las tarjetas individuales para que no se deformen y se mantengan dentro de la grilla?”
me respondio que modificando proyectos grid y las tarjetas para usar grid puro de manera robusta

adjunto foto de la respuesta:



CSS

```css
/* ========================================
   Opción A: CSS Grid en Proyectos
   ======================================== */
.proyectos-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 2rem;
}

.tarjeta-proyecto {
    background: white;
    border-radius: 0.8rem;
    overflow: hidden;
    box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
    transition: transform 0.3s ease, box-shadow 0.3s ease;
    display: flex;
    flex-direction: column; /* Alinea los elementos internos verticalmente */
    height: 100%;           /* Ocupa toda la altura de la celda de la grilla */
}
```

explicacion: despues de modificarlo en mi codigo para aplicarlo con la ayuda de la ia,
el display grid convierte el contenedor en una grilla bidimensional

`grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));` crea columnas automaticas

auto-fit ajusta la cantidad de columnas segun el ancho de la pantalla

minmax (280px, lfr) hace que cada tarjeta mida como minimo 280pr y que crezca lfr ocupando el espacio sin romper el diseño

display: flex; flex-direction: column; height: 100%;: hace que todas las tarjetas en la grilla esten en su lugar

despues para la opcion de mejora c la api publica con fetch y async/await
mi prompt inicial en la ia gemini fue

"quiero agregar una sección en mi js que consuma una api pública usando async/await y fetch necesito que tenga carga (cargando...) y errores visibles en la interfaz si la red falla”

la respuesta del chat fue “Una función asíncrona con bloques try...catch y manipulación del DOM para mostrar estados. “

despues en mi prompt de interacción le dije: "como puedo integrarlo visualmente en mi index.html actual y controlarlo desde el dom cuando cargue la pagina (domcontentloaded)"

con sus respuestas modifique mi codigo:
primero con
async/await que permite manejar operaciones asíncronas de javascript de forma limpia y secuencial esperando a que la red responda antes de continuar

fetch: realiza la petición http a la api externa para conseguir los datos en formato json

try...catch: bloque de control de errores fundamental para capturar fallas de conexión o de servidor y evitar que la aplicación se rompa

manejo de estados visuales: oculta y muestra elementos del dom (apiloading, apicontent, apierror) para darle una experiencia de usuario fluida y profesional.

en el nivel 4 de bonus elegi integrar el
prefers-color-scheme con el toogle modo oscuro
con la ayuda de la ia
prompt: necesito integrar prefers-color-scheme con el toogle de modo oscuro

respuesta de la ia: Paso 1: Actualizar la lógica en script.js

Reemplazá tu bloque inicial de detección de tema dentro del DOMContentLoaded en script.js por este código actualizado:

JavaScript

```
const toggleBtn = document.getElementById("toggle-tema");
const body = document.body;

// 1. Verificamos si hay una preferencia guardada en localStorage
const temaGuardado = localStorage.getItem("tema");

// 2. Verificamos si el sistema operativo del usuario prefiere modo oscuro
const prefiereOscuroSistema = window.matchMedia("(prefers-color-scheme: dark)").
// Lógica de activación inicial:
// Tiene prioridad lo que el usuario guardó manualmente, si no hay nada, usa la
if (temaGuardado === "oscuro" || (!temaGuardado && prefiereOscuroSistema)) {
    body.classList.add("dark-mode");
    toggleBtn.textContent = "☀️";
} else {
    toggleBtn.textContent = "🌙";
}

// Evento del botón toggle (modo manual)
toggleBtn.addEventListener("click", () => {
    body.classList.toggle("dark-mode");

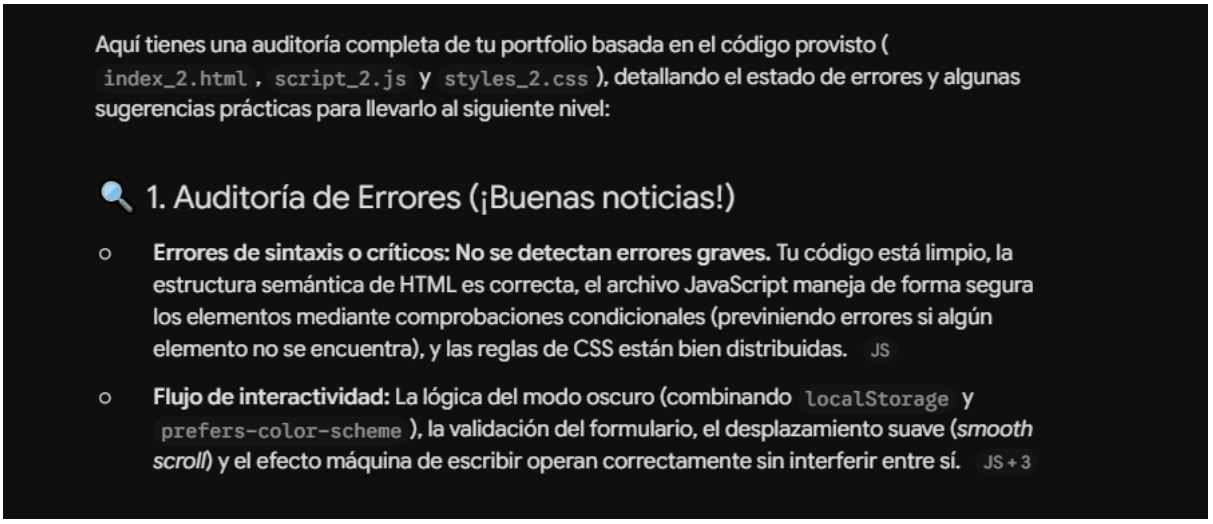
    const esOscuro = body.classList.contains("dark-mode");

    if (esOscuro) {
        toggleBtn.textContent = "☀️";
        localStorage.setItem("tema", "oscuro");
    } else {
        toggleBtn.textContent = "🌙";
        localStorage.setItem("tema", "claro");
    }
});
```

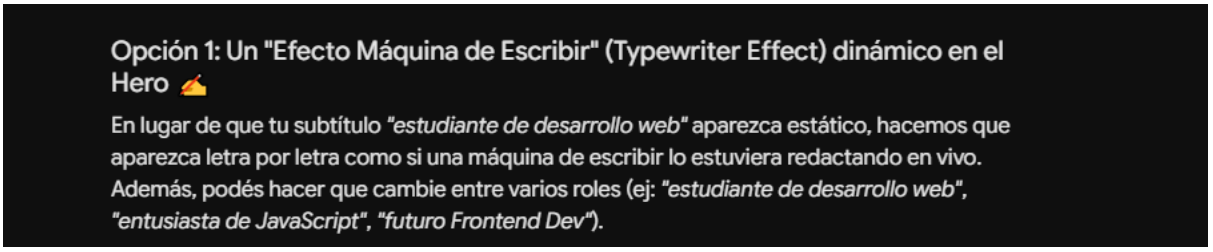
con la ayuda de la ia modifique mi codigo para agregar lo que se me pide

agregue la consulta de medios window.matchMedia('(prefers-color-scheme: dark)') con localstorage esto permite que el portfolio detecte automaticamente si el sistema operativo del usuario utiliza el modo oscuro por defecto cuando entra por primera vez, respeta la interactividad manual del botón toggle guardando la preferencia elegida, combinando la accesibilidad automática del sistema con la personalización del usuario

Le pedi a la ia que haga una auditoria completa y me de sugerencias de mejora



Ademas me dio varias opciones de mejora adicionales para el bonus de “Agregá una característica creativa bien implementada y explicada.”



asi que con la ayuda de la ia elegi esta opcion para el bonus de una caracteristica creativa
añadi un efecto de maquina de escribir basado en funciones asíncronas y temporizadores dinamicos (settimeout) el script manipula un arreglo

de cadenas de texto y controla iterativamente los índices de caracteres (charindex) mediante un ciclo booleano (estaborrando). esto permtie simular el efecto de maquina de escribir dentro del dom, complementado visualmente en css con una animación de fotogramas clave (@keyframes) para el parpadeo del cursor, mejorando notablemente el dinamismo y la retención visual de la sección de presentación (hero)